

# CHAPTER 1- INTRODUCTION

## 1.1 Introduction

The rapid advancement of Artificial Intelligence (AI), Machine Learning (ML), and Large Language Models (LLMs) has transformed the way complex engineering systems are monitored, analyzed, and maintained. Modern aerospace and defense industries generate massive amounts of operational data during engine testing, making manual analysis increasingly difficult and time-consuming. Engineers often spend significant effort examining telemetry data, identifying abnormal patterns, calculating performance metrics, and consulting technical manuals before making engineering decisions.

In propulsion engine testing environments, hundreds of sensor readings are generated continuously during each test cycle. These sensor values include parameters such as Exhaust Gas Temperature (EGT), Fuel Flow Rate, Rotational Speed (RPM), Torque, Pressure Ratios, and Vibration Levels. Traditional analysis methods require engineers to manually inspect these values and compare them against predefined operational limits. This process can be slow, labor-intensive, and prone to human error.

To address these challenges, an intelligent software platform named AI-Based Propulsion Test Analysis Platform has been developed. The platform combines modern web technologies, telemetry analysis, machine learning techniques, and Retrieval Augmented Generation (RAG) architecture to automate propulsion engine analysis. The system enables engineers to upload telemetry data, analyze engine performance, visualize operational trends, retrieve engineering knowledge, and receive AI-assisted recommendations through a single integrated dashboard.

The project aims to improve analytical efficiency, reduce manual effort, and provide faster access to technical insights while maintaining data security through local deployment. By integrating AI capabilities into propulsion testing workflows, the platform demonstrates how modern intelligent systems can enhance aerospace engineering operations.

## 1.2 Background of Aerospace Propulsion Engine Testing

### Importance of Propulsion Systems

A propulsion system is one of the most critical components of any aerospace vehicle. It generates the thrust required to move aircraft, missiles, spacecraft, and defense platforms. The performance, efficiency, and reliability of propulsion systems directly influence mission success, operational safety, and overall system effectiveness.

Before deployment, propulsion systems undergo extensive testing to verify their performance under various operating conditions. These tests generate large volumes of telemetry data that provide valuable insights into engine behavior. Engineers analyze this data to ensure that engines operate within acceptable performance limits and meet design specifications.

Modern propulsion systems contain sophisticated sensors capable of measuring numerous operational parameters in real time. The resulting datasets often contain thousands of records, making manual analysis increasingly challenging. As aerospace systems become more advanced, there is a growing need for intelligent analytical tools capable of processing large-scale telemetry information efficiently.

### Challenges in Traditional Engine Testing

Traditional propulsion testing methods rely heavily on manual analysis performed by domain experts. Although these methods have been effective for many years, they face several limitations when dealing with modern high-volume telemetry datasets.

Engineers must often spend considerable time identifying abnormal sensor readings, calculating performance indicators, generating visualizations, and preparing technical reports. These repetitive tasks reduce productivity and increase the possibility of overlooking subtle anomalies hidden within the data.

Another challenge is the difficulty of accessing technical knowledge during analysis. Engineers frequently consult maintenance manuals, technical standards, and historical reports while investigating anomalies. Searching through large document collections manually can significantly delay decision-making.

As a result, aerospace organizations increasingly seek intelligent systems capable of automating analytical workflows and providing context-aware technical assistance.

**Table 1.1: Challenges in Traditional Propulsion Testing**

| Sl. No | Challenge               | Description                               | Impact                   |
|--------|-------------------------|-------------------------------------------|--------------------------|
| 1      | Manual Data Analysis    | Engineers manually inspect telemetry data | Increased analysis time  |
| 2      | Large Data Volume       | Thousands of sensor readings generated    | Difficult interpretation |
| 3      | Delayed Fault Detection | Hidden anomalies may remain unnoticed     | Reduced reliability      |
| 4      | Report Preparation      | Documentation performed manually          | Increased workload       |
| 5      | Knowledge Retrieval     | Manual search of technical documents      | Slower decision-making   |

**Explanation:**

The above table highlights major limitations associated with traditional propulsion testing methodologies. These challenges motivate the adoption of AI-based analytical platforms.

**1.3 Need for Intelligent Engine Analysis Systems**

The increasing complexity of aerospace propulsion systems has created a strong demand for intelligent analytical solutions. Modern testing environments generate significantly more data than engineers can efficiently process through manual methods alone.

An intelligent analysis platform can automatically process telemetry data, calculate performance metrics, detect abnormal behavior, and generate reports with minimal human intervention. Such systems improve operational efficiency while allowing engineers to focus on higher-level decision-making tasks.

Furthermore, advances in Artificial Intelligence, Machine Learning, and Natural Language Processing (NLP) have made it possible to create engineering assistants capable of understanding technical information and providing meaningful recommendations based on historical knowledge.

The integration of AI into propulsion testing therefore represents a significant step toward improving reliability, accuracy, and productivity within aerospace engineering environments.

**1.4 Problem Statement**

Propulsion engine testing generates large amounts of telemetry data that require extensive manual analysis. Engineers must inspect sensor values, calculate performance metrics, identify anomalies, consult technical documentation, and prepare reports before drawing conclusions regarding engine health.

This traditional workflow consumes significant time and effort while increasing the possibility of human error. Additionally, accessing relevant technical information during anomaly investigation often requires manual document searches, further slowing the analytical process.

Therefore, there is a need for an intelligent software platform capable of automating telemetry analysis, supporting anomaly detection, retrieving engineering knowledge, and assisting engineers with technical recommendations in a secure environment.

**1.5 Objectives of the Project**

The primary objective of this project is to develop an intelligent propulsion analysis platform capable of automating major activities involved in engine testing and performance evaluation.

Specific objectives include:

- To develop a web-based dashboard for propulsion analysis.

- To support telemetry data upload through CSV files.
- To automate engine performance analysis.
- To calculate important operational metrics.
- To visualize engine behavior using graphical representations.
- To maintain a repository of historical test records.
- To implement Retrieval Augmented Generation (RAG) for knowledge retrieval.
- To integrate a locally deployed AI assistant.
- To generate automated engineering reports.
- To improve efficiency and reduce manual analytical effort.

**Table 1.2: Project Objectives and Expected Outcomes**

| Objective              | Expected Outcome                   |
|------------------------|------------------------------------|
| Telemetry Processing   | Automated sensor data analysis     |
| Performance Evaluation | Faster engine health assessment    |
| AI Integration         | Intelligent engineering assistance |
| RAG Implementation     | Improved knowledge retrieval       |
| Dashboard Development  | Enhanced user experience           |
| Report Generation      | Reduced documentation effort       |

## 1.6 Scope of the Project

The scope of this project is focused on developing an integrated software platform for propulsion engine telemetry analysis. The system accepts telemetry data in CSV format and processes the information to generate meaningful analytical insights.

The platform supports performance monitoring, visualization, AI-assisted analysis, document retrieval, and report generation. The project emphasizes secure local deployment to ensure that sensitive engineering information remains protected within organizational infrastructure.

Although the current implementation focuses on propulsion engine testing, the architecture can be extended in the future to support other aerospace applications such as structural health monitoring, avionics diagnostics, and predictive maintenance systems.

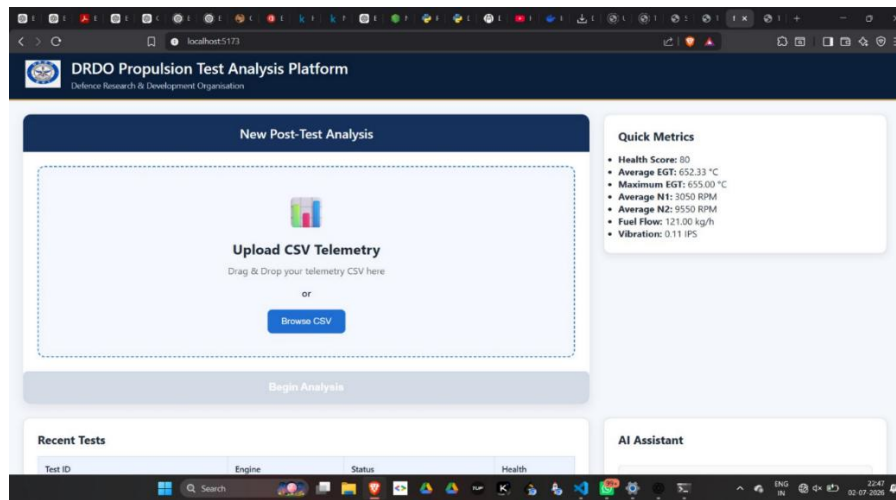
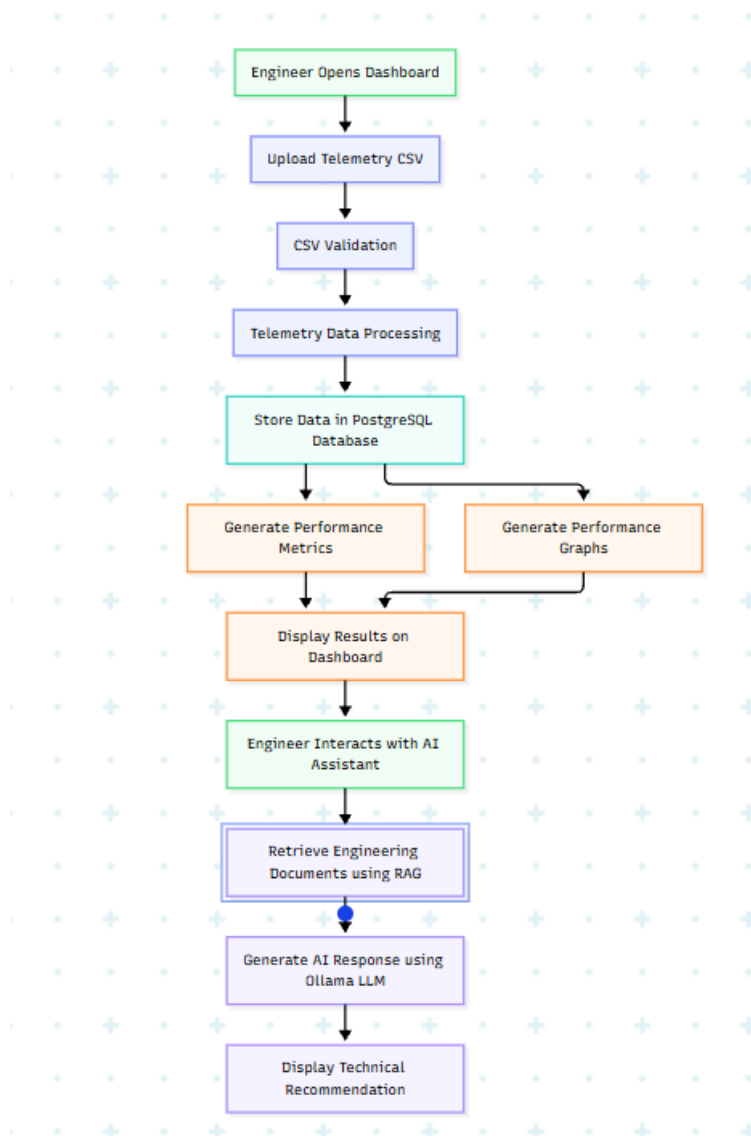


Figure1.6: Main Dashboard of the DRDO propulsion Test Analysis



Flowchart 1 – Overall Workflow of the AI – Based Propulsion Test Analysis Platform

## **CHAPTER 2 - LITERATURE SURVEY AND EXISTING SYSTEM**

### **2.1 Introduction**

The development of Artificial Intelligence (AI) has significantly influenced modern engineering applications, particularly in the fields of aerospace, defense, and industrial automation. Propulsion engine testing has traditionally relied on manual observation and engineering expertise to evaluate engine performance. However, with increasing engine complexity and the growing volume of telemetry data, traditional analysis techniques have become less efficient.

A Literature Survey provides an understanding of previously developed methods, technologies, and research contributions related to propulsion engine monitoring and intelligent data analysis. Studying existing systems helps identify their strengths and limitations while providing a strong foundation for designing an improved solution.

The proposed AI-Based Propulsion Test Analysis Platform has been developed after studying current telemetry analysis methods, machine learning applications, Retrieval Augmented Generation (RAG), and AI-assisted engineering systems. The objective is to overcome the limitations of conventional propulsion testing by introducing intelligent automation, interactive visualization, and context-aware engineering assistance.

This chapter discusses traditional propulsion testing methods, existing engine health monitoring systems, the role of Artificial Intelligence in aerospace engineering, limitations of conventional approaches, and the advantages offered by the proposed system.

### **2.2 Traditional Propulsion Engine Testing**

#### **Overview**

Propulsion engines are critical components of aircraft, missiles, and spacecraft. Before deployment, every engine undergoes extensive testing to verify its operational performance, reliability, and safety. During testing, numerous sensors continuously monitor important parameters such as Exhaust Gas Temperature (EGT), Fuel Flow, Rotational Speed (RPM), Torque, Pressure, and Vibration.

Traditionally, these measurements are collected through telemetry systems and stored for later analysis. Engineers manually inspect the recorded data to determine whether the engine operates within acceptable limits.

#### **Working Principle**

Traditional engine testing generally follows a sequence of predefined activities:

- Installation of sensors on the propulsion engine.

- Execution of engine test runs.
- Collection of telemetry data.
- Manual inspection of sensor values.
- Comparison with engineering standards.
- Preparation of technical reports.

Although this procedure has been used successfully for many years, it becomes increasingly difficult as telemetry datasets continue to grow in size.

### Limitations of Manual Analysis

Manual data analysis introduces several operational challenges. Engineers must examine thousands of sensor readings individually, making the process time-consuming and susceptible to human error. Small abnormalities may remain unnoticed, especially when hidden among large datasets.

Another challenge is the preparation of engineering reports. Since calculations, graphs, and observations are generated manually, documentation requires considerable effort.

Furthermore, engineers often need to consult maintenance manuals and historical reports while investigating anomalies. Searching through technical documents manually slows down decision-making and increases analysis time.

**Table 2.1: Characteristics of Traditional Propulsion Testing**

| Parameter           | Traditional Method     |
|---------------------|------------------------|
| Data Processing     | Manual                 |
| Graph Generation    | Manual                 |
| Fault Detection     | Engineer Observation   |
| Knowledge Retrieval | Manual Document Search |
| Report Generation   | Manual                 |
| Analysis Speed      | Slow                   |
| Human Error         | High Possibility       |

#### Explanation:

The table illustrates that conventional propulsion testing depends heavily on manual engineering effort. Most activities require human intervention, increasing both workload and analysis time.

## 2.3 Existing Engine Health Monitoring Systems

Modern aerospace organizations employ **Engine Health Monitoring (EHM)** systems to continuously monitor propulsion engine performance. These systems collect telemetry data and provide engineers with operational insights regarding engine condition.

Existing monitoring systems typically display sensor values, generate graphs, and issue warnings whenever parameters exceed predefined limits. While these capabilities improve operational awareness, they still depend largely on threshold-based monitoring rather than intelligent analysis.

Most existing platforms are designed primarily for data visualization and do not provide advanced AI capabilities such as contextual engineering recommendations or semantic knowledge retrieval.

Another limitation is that many commercial monitoring systems rely on cloud-based infrastructure. In defense environments where sensitive operational data must remain secure, cloud deployment may not always be appropriate.

Consequently, there is a growing need for locally deployed intelligent systems capable of performing advanced analytical functions while maintaining complete data confidentiality.

## 2.4 Artificial Intelligence in Aerospace Engineering

The integration of **Artificial Intelligence** into aerospace engineering has transformed many engineering activities including predictive maintenance, fault diagnosis, autonomous navigation, manufacturing automation, and propulsion analysis.

AI systems can analyze large datasets much faster than traditional manual methods. Machine learning algorithms identify hidden relationships between sensor values and detect subtle anomalies that may not be visible through conventional threshold monitoring.

In recent years, **Large Language Models (LLMs)** have introduced new possibilities for engineering knowledge assistance. Engineers can now interact with intelligent assistants capable of understanding technical questions and retrieving relevant information from engineering documentation.

The combination of AI, telemetry analysis, and document retrieval has significantly improved engineering productivity and reduced analysis time across many aerospace applications.

## 2.5 Literature Review of Related Technologies

Several research studies have demonstrated the benefits of AI in propulsion engine diagnostics and aerospace maintenance.



Researchers have successfully applied **Machine Learning** algorithms for anomaly detection using telemetry datasets collected from gas turbine engines. These approaches improve early fault detection and reduce unexpected maintenance events.

Other studies have explored **Retrieval Augmented Generation (RAG)** for technical document retrieval. Instead of depending solely on pretrained language models, RAG combines semantic document retrieval with AI reasoning to produce technically accurate responses.

Recent developments in **Local Large Language Models** have enabled organizations to deploy AI systems within secure infrastructure without transmitting sensitive engineering data to external cloud services.

These technological developments directly influenced the design of the proposed AI-Based Propulsion Test Analysis Platform.

## **2.6 Limitations of Existing Systems**

Although current propulsion analysis systems provide useful monitoring capabilities, several important limitations remain.

### **Manual Interpretation**

Engineers must still interpret graphs and telemetry data manually before making engineering decisions.

### **Limited Intelligence**

Most systems provide only threshold-based warnings instead of intelligent anomaly analysis.

### **Lack of Knowledge Integration**

Existing monitoring platforms usually cannot retrieve engineering information from manuals or historical reports automatically.

### **Cloud Dependency**

Many AI solutions depend on internet connectivity and cloud infrastructure, making them unsuitable for secure defense applications.

### **Time-Consuming Documentation**

Technical reports are often prepared manually after each testing session.

These limitations motivated the development of a more intelligent analytical platform.

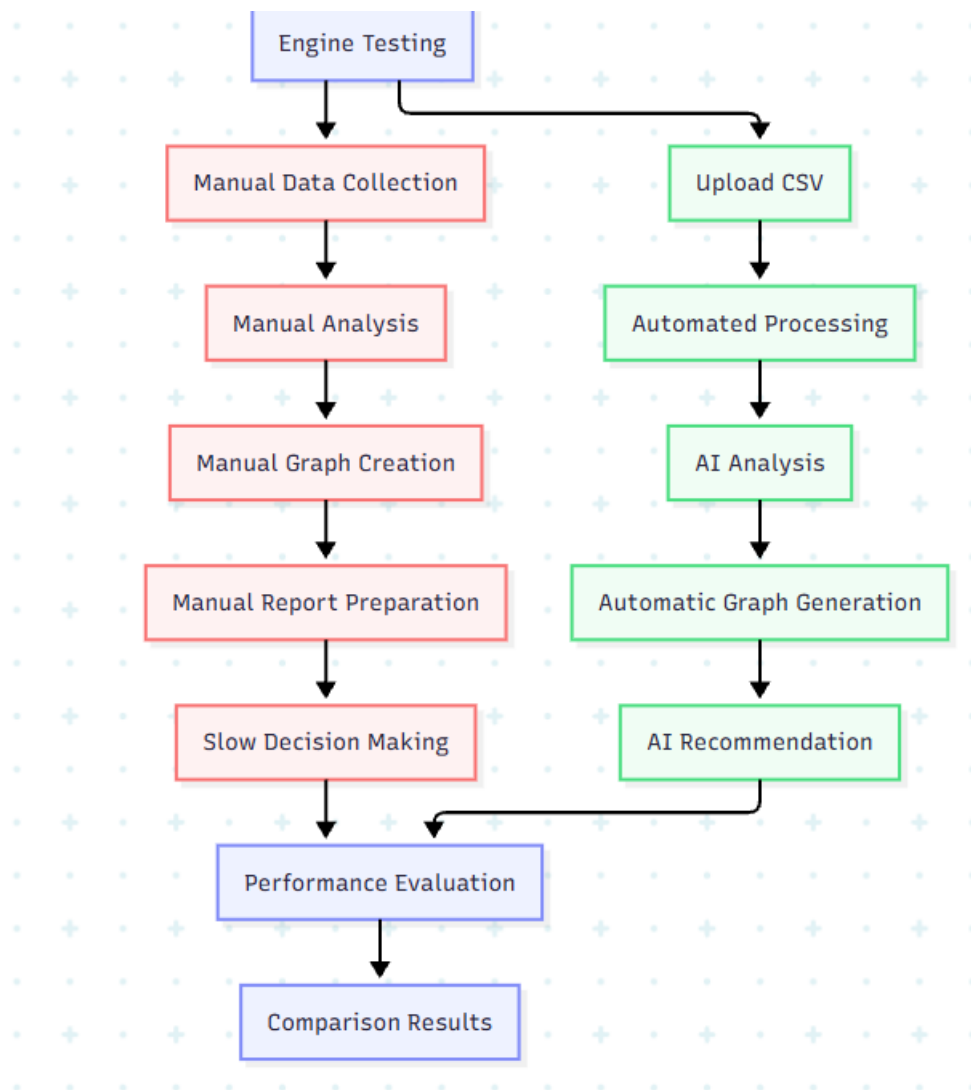
## **2.7 Proposed AI-Based Solution**

The proposed system integrates modern software technologies to automate propulsion engine analysis from data upload to final report generation.

Telemetry data is uploaded through a **ReactJS** dashboard and processed by a **Django REST Framework** backend. Performance metrics are calculated automatically using Python modules. Historical test records are maintained inside a **PostgreSQL** database.

Engineering manuals are converted into vector embeddings using **Chroma Vector Database**, allowing semantic document retrieval through **LangChain**. A locally deployed **Ollama Large Language Model** generates engineering recommendations using Retrieval Augmented Generation.

By combining these technologies, the proposed platform provides a secure, intelligent, and user-friendly environment for propulsion engine testing.



Flowchart 2 – Comparison of Traditional and AI-Based Propulsion Engine Analysis

## **CHAPTER 3 - TECHNOLOGIES USED**

### **3.1 Introduction**

The successful development of an intelligent software application depends on selecting appropriate technologies that provide high performance, reliability, scalability, and ease of maintenance. Since the proposed project integrates Artificial Intelligence, Machine Learning, Web Development, Database Management, and Retrieval Augmented Generation (RAG), multiple software technologies have been combined to create a unified engineering platform.

The AI-Based Propulsion Test Analysis Platform was developed using a modern full-stack architecture. The frontend of the application was designed using ReactJS, while the backend services were implemented using Python and Django REST Framework. The system stores telemetry data and historical records in PostgreSQL, processes telemetry datasets using Pandas, generates graphs using Matplotlib, and implements intelligent document retrieval using LangChain, ChromaDB, and Ollama.

Each technology plays a unique role in ensuring that the system performs efficiently while maintaining modularity and future scalability. This chapter discusses each technology in detail, explaining why it was selected and how it contributes to the overall functionality of the proposed system.

### **3.2 Python Programming Language**

#### **Introduction**

Python is one of the most widely used programming languages in modern software development because of its simplicity, flexibility, and extensive collection of libraries. It is especially popular in Artificial Intelligence, Machine Learning, Data Science, and Backend Web Development.

The proposed system uses Python as the primary programming language for backend development and telemetry data processing. Python provides an easy-to-understand syntax, allowing developers to build complex analytical applications with minimal code complexity. Its large ecosystem of libraries makes it ideal for engineering applications involving numerical computation, database connectivity, machine learning, and report generation.

#### **Role in the Project**

In this project, Python performs several critical operations:

- Processing uploaded telemetry CSV files.
- Calculating engine performance metrics.
- Generating analytical graphs.

- Communicating with the PostgreSQL database.
- Implementing AI modules.
- Integrating LangChain and ChromaDB.
- Running backend APIs through Django.

Because of its versatility, Python serves as the backbone of the entire application.

### **Advantages of Python**

- Simple syntax suitable for rapid development.
- Extensive AI and Machine Learning libraries.
- Strong community support.
- Excellent database integration.
- Cross-platform compatibility.
- High scalability.

## **3.3 ReactJS**

### **Introduction**

ReactJS is an open-source JavaScript library used for developing interactive and responsive user interfaces. It follows a component-based architecture where each section of the webpage is developed as an independent reusable component.

For this project, ReactJS was selected to build the frontend dashboard because it provides a modern, fast, and highly interactive user experience.

### **Role in the Project**

The React frontend allows engineers to interact with the system through a clean dashboard. Users can upload telemetry CSV files, view engine performance metrics, access historical test records, communicate with the AI assistant, and visualize engine graphs.

React components developed in the project include:

- Header Component
- Dashboard Component
- Upload Module
- Repository Table
- Graph Display
- AI Assistant Panel

Its component-based architecture improves maintainability and allows future feature additions without affecting existing modules.

**Table 3.1: Features of ReactJS**

| Feature                      | Benefit                    |
|------------------------------|----------------------------|
| Component-Based Architecture | Reusable code              |
| Virtual DOM                  | Faster rendering           |
| State Management             | Efficient UI updates       |
| Responsive Interface         | Better user experience     |
| API Integration              | Easy backend communication |

### 3.4 Django REST Framework

#### Introduction

Django REST Framework (DRF) is a powerful backend framework used for developing secure RESTful APIs. It simplifies backend development by providing built-in support for authentication, serialization, request handling, and database interaction.

The backend of the proposed system was implemented using Django REST Framework because it offers high reliability and integrates seamlessly with Python-based AI libraries.

#### Role in the Project

Django performs several important functions:

- Receives uploaded CSV files.
- Processes API requests.
- Stores telemetry information.
- Communicates with PostgreSQL.
- Returns processed results to ReactJS.
- Supports AI module integration.

The framework acts as the communication bridge between the frontend dashboard and backend analytical modules.

### **3.5 PostgreSQL Database**

#### **Introduction**

PostgreSQL is a powerful open-source relational database management system widely used for enterprise-level applications. It offers excellent performance, reliability, security, and support for complex SQL operations.

In the proposed platform, PostgreSQL stores all uploaded telemetry records, historical engine test information, user-generated reports, and analytical results.

#### **Role in the Project**

The database performs several important tasks:

- Stores uploaded telemetry datasets.
- Maintains historical test records.
- Stores generated reports.
- Retrieves previous engine tests.
- Supports repository management.

Using PostgreSQL ensures that analytical data remains organized and easily retrievable.

### **3.6 Pandas Library**

Pandas is a Python library used for handling structured datasets. It provides powerful functions for reading, cleaning, filtering, transforming, and analyzing large CSV files.

Within this project, Pandas is responsible for reading uploaded telemetry CSV files and converting them into structured dataframes for further analysis. Missing values, incorrect formats, and duplicate records are handled efficiently before calculations begin.

### **3.7 Matplotlib**

Matplotlib is a visualization library used for creating engineering graphs and performance charts.

The project uses Matplotlib to generate graphs showing:

- Engine RPM
- Fuel Flow
- Exhaust Gas Temperature
- Torque

- Vibration

Graphical visualization enables engineers to interpret engine behaviour much more efficiently than numerical values alone.

### 3.8 LangChain Framework

LangChain is an application development framework used for integrating Large Language Models with external data sources. It enables the system to retrieve engineering documents and provide context-aware AI responses.

In the proposed project, LangChain connects the vector database with the local AI assistant.

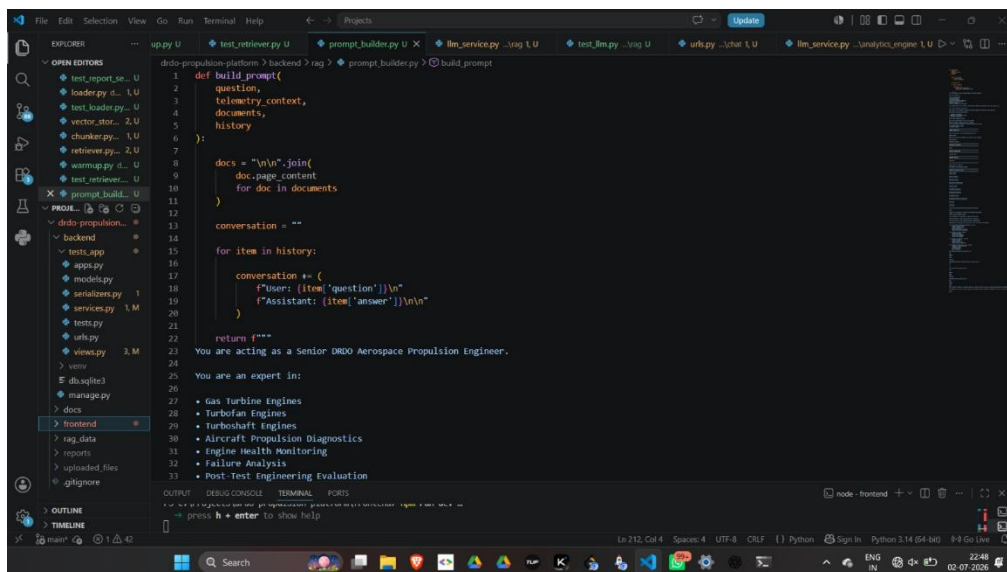
Its major functions include:

- Document Retrieval
- Prompt Construction
- Context Generation
- AI Response Management

### 3.9 Chroma Vector Database

The project uses ChromaDB to store vector embeddings generated from engineering manuals and technical documents. Instead of performing traditional keyword searches, ChromaDB enables semantic search based on contextual meaning.

Whenever the AI assistant receives a query, ChromaDB retrieves the most relevant engineering document sections before sending them to the language model.



**Figure 3.9: Prompt Builder for the Large Language Model**

### **3.10 Ollama Local Large Language Model**

Ollama enables the project to run Large Language Models locally without requiring cloud connectivity. This is particularly important in aerospace and defense environments where engineering data must remain secure.

The local AI assistant uses Ollama together with LangChain and ChromaDB to answer technical questions based on uploaded engineering documents.

Running the model locally provides:

- Enhanced security
- Offline operation
- Faster response time
- Protection of confidential engineering data

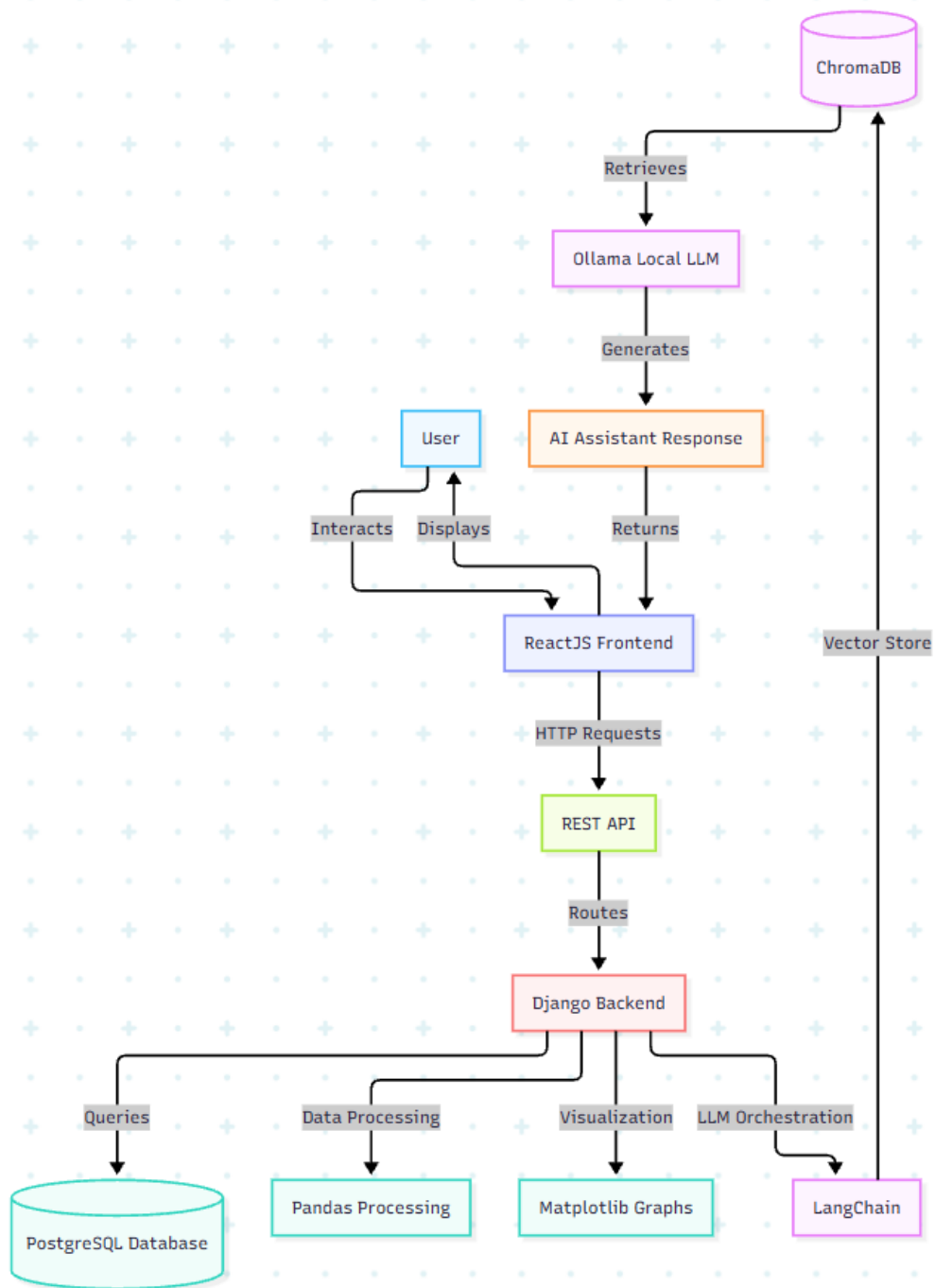
### **3.11 Retrieval Augmented Generation (RAG)**

The proposed platform implements Retrieval Augmented Generation (RAG) to improve AI response quality.

Instead of answering questions using only pretrained knowledge, the system first retrieves relevant engineering documents from the vector database. These retrieved documents are then supplied to the local language model, allowing it to generate technically accurate and context-aware responses.

This architecture significantly improves engineering decision support and ensures that AI recommendations remain relevant to uploaded technical documentation.





Flowchart 3 – Technology Stack of the Proposed System

## **CHAPTER 4 - SYSTEM ARCHITECTURE AND DESIGN**

### **4.1 Introduction**

The System Architecture defines the overall structure of the proposed AI-Based Propulsion Test Analysis Platform and illustrates how different software components interact to perform telemetry analysis, engine health monitoring, and intelligent recommendation generation. A well-designed architecture is essential for ensuring scalability, maintainability, security, and efficient communication between the frontend, backend, database, and Artificial Intelligence modules.

The proposed system follows a modular architecture, where each module performs a specific function while remaining connected to the others through secure APIs. This design makes the application easier to maintain and allows new features to be added without affecting the existing functionality.

The platform consists of several integrated components, including the ReactJS frontend, Django REST Framework backend, PostgreSQL database, CSV processing module, graph visualization module, Retrieval Augmented Generation (RAG) module, Chroma Vector Database, and the Ollama Local Large Language Model. Together, these components automate the complete workflow of propulsion engine telemetry analysis.

The architecture ensures that engineers can upload telemetry data, process it automatically, retrieve historical records, visualize engine performance, and interact with an AI assistant through a single dashboard. Since the AI model runs locally using Ollama, the platform maintains data confidentiality, making it suitable for environments where sensitive engineering information must remain secure.

### **4.2 Overall System Architecture**

#### **Overview**

The proposed system follows a three-tier architecture consisting of the Presentation Layer, Application Layer, and Data Layer.

The Presentation Layer is responsible for user interaction. It provides a responsive dashboard where engineers can upload telemetry data, view analytical graphs, access historical test records, and communicate with the AI assistant.

The Application Layer contains the business logic of the system. It processes uploaded CSV files, calculates engine performance metrics, manages communication between different modules, and generates analytical results.

The Data Layer stores telemetry datasets, historical records, vector embeddings, and engineering documents. This layer ensures efficient retrieval of information while maintaining data integrity.

The architecture enables seamless communication between all software components, resulting in faster data processing and improved analytical performance.

### Main Components of the Architecture

The architecture consists of the following major modules:

- ReactJS Frontend
- Django REST Backend
- Telemetry Processing Module
- Graph Visualization Module
- PostgreSQL Database
- RAG Knowledge Base
- Chroma Vector Database
- Ollama Local LLM
- AI Engineering Assistant

Each module performs a dedicated function while communicating through REST APIs.

**Table 4.1: Major System Components**

| Component             | Function                            |
|-----------------------|-------------------------------------|
| ReactJS               | User Interface Development          |
| Django REST Framework | Backend API Services                |
| PostgreSQL            | Stores Telemetry Data               |
| Pandas                | CSV Data Processing                 |
| Matplotlib            | Performance Graph Generation        |
| ChromaDB              | Stores Vector Embeddings            |
| LangChain             | Connects AI with Knowledge Base     |
| Ollama                | Executes Local Large Language Model |

**Explanation:**

The table summarizes the primary software components used in the proposed architecture and their individual responsibilities.

### 4.3 Frontend Architecture

**Overview**

The frontend serves as the interaction layer between the user and the software platform. It was developed using ReactJS, which provides a responsive, component-based architecture capable of updating the interface dynamically without reloading the webpage.

The dashboard is designed to simplify the engine analysis process by presenting all required functions within a single interface. Engineers can upload telemetry CSV files, monitor engine performance metrics, visualize graphical trends, review historical test records, and communicate with the AI assistant.

ReactJS components are reusable, making future development easier and improving software maintainability.

**Frontend Modules**

The frontend consists of several independent modules:

- Dashboard
- Navigation Header
- CSV Upload Section
- Recent Tests Repository
- Quick Metrics Panel
- AI Assistant Panel
- Graph Visualization Section

Each module is responsible for presenting a specific category of information to the user.

### 4.4 Backend Architecture

**Overview**

The backend forms the processing core of the proposed platform. It was developed using Python and Django REST Framework, which provide secure RESTful APIs for communication between the frontend and backend services.

Whenever a telemetry CSV file is uploaded, the backend validates the file, extracts telemetry values, stores the information in PostgreSQL, calculates engine performance metrics, and returns processed results to the dashboard.

The backend also communicates with the AI modules to provide engineering recommendations.

### **Responsibilities of Backend**

The backend performs several important functions:

- Receives uploaded CSV files.
- Validates uploaded data.
- Stores telemetry information.
- Generates engine performance metrics.
- Creates graphs.
- Retrieves historical records.
- Executes AI assistant requests.

## **4.5 Database Architecture**

### **Overview**

The proposed platform uses PostgreSQL as its relational database system. The database stores uploaded telemetry files, engine performance parameters, historical test records, and generated reports.

A structured database ensures efficient retrieval of information while maintaining consistency and reliability.

The database also supports future expansion by allowing additional tables to be integrated without affecting existing records.

### **Database Stores**

The PostgreSQL database contains:

- User Information
- Telemetry Records
- Engine Parameters
- Historical Tests
- Reports
- AI Queries

**Table 4.2: Database Entities**

| Entity     | Description           |
|------------|-----------------------|
| User       | Login Information     |
| Test       | Uploaded Engine Tests |
| Telemetry  | Sensor Values         |
| Reports    | Generated Reports     |
| Repository | Historical Records    |

#### 4.6 API Communication

Communication between the frontend and backend is performed through REST APIs.

The React dashboard sends requests to the Django backend whenever the user uploads telemetry data or interacts with the AI assistant.

The backend processes these requests and returns structured JSON responses to the frontend.

This architecture enables efficient data exchange while keeping the frontend and backend independent.

##### API Operations

- CSV Upload API
- Historical Test API
- Graph Generation API
- AI Assistant API
- Repository API

#### 4.7 System Workflow

The overall workflow of the proposed platform consists of several sequential stages.

First, engineers upload telemetry CSV files through the dashboard. The backend validates the uploaded data and extracts telemetry parameters using Pandas. Performance metrics are calculated and stored in PostgreSQL.

The processed data is then visualized through interactive graphs. Historical records become available in the repository section.

When engineers ask technical questions, the AI assistant retrieves relevant engineering documents from ChromaDB using LangChain. These documents are supplied to the Ollama Local LLM, which generates context-aware engineering recommendations.

Finally, the dashboard displays analytical results and AI-generated responses.

#### **4.8 Advantages of the Proposed Architecture**

The modular architecture offers several advantages:

- High Scalability
- Easy Maintenance
- Fast Processing
- Secure Local Deployment
- Modular Design
- AI Integration
- Efficient Database Management
- Interactive User Interface
- Future Expandability

These features make the system suitable for handling large telemetry datasets while providing intelligent engineering assistance.

## **CHAPTER 5- AI-BASED ANALYSIS AND RETRIEVAL AUGMENTED GENERATION (RAG) IMPLEMENTATION**

### **5.1 Introduction**

Artificial Intelligence has emerged as one of the most influential technologies in modern engineering, enabling systems to analyze large datasets, identify hidden patterns, automate repetitive tasks, and assist engineers in making informed decisions. In aerospace and defense applications, AI has become increasingly valuable because propulsion engine testing generates vast amounts of telemetry data that require continuous monitoring and interpretation.

The proposed AI-Based Propulsion Test Analysis Platform integrates Artificial Intelligence to simplify propulsion engine analysis. Instead of requiring engineers to manually inspect telemetry data and search through technical manuals, the system automatically processes uploaded information and provides intelligent recommendations through an AI assistant.

A major feature of the platform is the implementation of Retrieval Augmented Generation (RAG). Unlike traditional AI chatbots that depend only on pre-trained knowledge, RAG enables the AI assistant to retrieve relevant information from engineering manuals and technical documents before generating responses. This approach improves the accuracy, reliability, and relevance of AI-generated answers.

To implement this capability, the project combines LangChain, ChromaDB, and Ollama into a unified architecture. These technologies enable the system to retrieve engineering knowledge, understand user queries, and generate technically meaningful responses while operating entirely offline.

### **5.2 Artificial Intelligence Integration**

#### **Introduction**

Artificial Intelligence forms the intelligence layer of the proposed system. It assists engineers by analyzing information, retrieving engineering knowledge, and generating recommendations based on uploaded technical documents.

Traditional software applications generally display numerical values or graphs but cannot explain why a particular engine parameter behaves abnormally. The proposed AI assistant overcomes this limitation by providing context-aware explanations.

For example, if an engineer asks:

**"Why is the Exhaust Gas Temperature high?"**

the AI assistant searches the engineering knowledge base, retrieves relevant document sections, and generates an explanation using the local Large Language Model.



This significantly reduces the time required for manual investigation while improving engineering productivity.

### Functions of AI in the Proposed System

Artificial Intelligence performs multiple tasks within the platform.

- Understanding engineering questions.
- Retrieving relevant technical information.
- Explaining abnormal telemetry behaviour.
- Assisting engine diagnostics.
- Supporting engineering decision-making.
- Improving analytical efficiency.

These capabilities transform the software from a simple dashboard into an intelligent engineering assistant.

**Table 5.1: AI Functions in the Proposed System**

| AI Function         | Purpose                         |
|---------------------|---------------------------------|
| Query Understanding | Understand engineer's question  |
| Knowledge Retrieval | Search engineering documents    |
| Response Generation | Generate technical explanations |
| Context Analysis    | Improve response accuracy       |
| Decision Support    | Assist engineering analysis     |

#### Explanation

The above table illustrates how Artificial Intelligence contributes to different stages of propulsion engine analysis. Instead of simply displaying numerical values, the AI assistant actively supports engineers by providing explanations and recommendations.

## 5.3 Retrieval Augmented Generation (RAG)

### Introduction

Retrieval Augmented Generation (RAG) is an advanced AI architecture that combines document retrieval with Large Language Models. Unlike conventional chatbots that rely only

on pre-trained knowledge, RAG first retrieves relevant information from external documents before generating responses.

This approach is particularly useful in engineering applications where technical accuracy is essential. Engineering manuals, maintenance guides, and historical reports often contain valuable information that should be considered during analysis.

By retrieving relevant documents before answering a question, RAG significantly improves the reliability and contextual relevance of AI-generated responses.

## **Working Principle**

The RAG process consists of several sequential steps.

First, engineering documents are converted into vector embeddings.

These embeddings are stored inside ChromaDB.

When an engineer asks a question, the system converts the query into an embedding and searches for similar vectors.

The retrieved document sections are then supplied to the Ollama Local Large Language Model.

Finally, the AI assistant generates a response based on both the retrieved information and its language understanding capabilities.

## **5.4 Knowledge Base Creation**

### **Introduction**

A knowledge base is a collection of engineering documents that provides reference information for the AI assistant.

The proposed platform allows technical manuals, PDF reports, and engineering reference documents to be uploaded into the system. These documents are processed and converted into searchable vector embeddings.

Instead of performing traditional keyword searches, semantic embeddings enable the AI assistant to understand the meaning of engineering content.

This greatly improves the quality of retrieved information.

### **Knowledge Base Workflow**

The knowledge base performs the following operations.

- Document Upload

- Text Extraction
- Document Chunking
- Embedding Generation
- Vector Storage
- Semantic Retrieval

These operations ensure that engineering knowledge becomes easily accessible during AI conversations.

**Table 5.2: Knowledge Base Processing Steps**

| Step                 | Description                   |
|----------------------|-------------------------------|
| Document Upload      | Upload engineering manuals    |
| Text Extraction      | Read PDF content              |
| Chunk Creation       | Divide document into sections |
| Embedding Generation | Convert text into vectors     |
| Vector Storage       | Store inside ChromaDB         |
| Semantic Search      | Retrieve relevant information |

## 5.5 Vector Embedding Generation

Vector embeddings are numerical representations of text that capture semantic meaning.

Instead of comparing exact words, vector embeddings allow the system to identify documents with similar meaning.

For example, the questions:

- "Why is EGT high?"
- "Reason for increased Exhaust Gas Temperature"

produce similar embeddings and therefore retrieve the same engineering information.

This capability makes document retrieval much more intelligent than traditional keyword searching.

## 5.6 Chroma Vector Database

The project uses ChromaDB as the vector database responsible for storing embeddings generated from engineering documents.

Unlike relational databases, ChromaDB stores vectors instead of tables.

Whenever an engineer submits a query, ChromaDB performs similarity search and retrieves the most relevant document sections.

This significantly improves response quality while reducing search time.

### Advantages of ChromaDB

- Fast semantic search.
- Efficient document retrieval.
- High scalability.
- Easy integration with LangChain.
- Supports Retrieval Augmented Generation.

## 5.7 Prompt Engineering

Prompt Engineering refers to designing effective prompts that guide the Large Language Model toward generating accurate responses.

Instead of sending only the user's question, the system combines:

- Retrieved document context
- Engineering instructions
- User query

This combined prompt enables the AI assistant to produce technically accurate recommendations.

Carefully designed prompts reduce hallucinations and improve engineering reliability.

## 5.8 LangChain Framework

LangChain acts as the central controller connecting ChromaDB with the Local Large Language Model.

Its responsibilities include:

- Document Retrieval
- Prompt Construction

- AI Pipeline Management
- Context Injection
- Response Generation

Without LangChain, integrating document retrieval with AI reasoning would require significantly more custom programming.

## 5.9 Ollama Local Large Language Model

The proposed platform uses Ollama to execute the Large Language Model locally.

Unlike cloud-based AI services, Ollama runs entirely on the local computer.

This provides several important advantages.

### Advantages

- Complete offline operation.
- Data confidentiality.
- Faster response time.
- No internet dependency.
- Suitable for defense applications.

Running the AI locally ensures that sensitive propulsion engine information never leaves the organization.

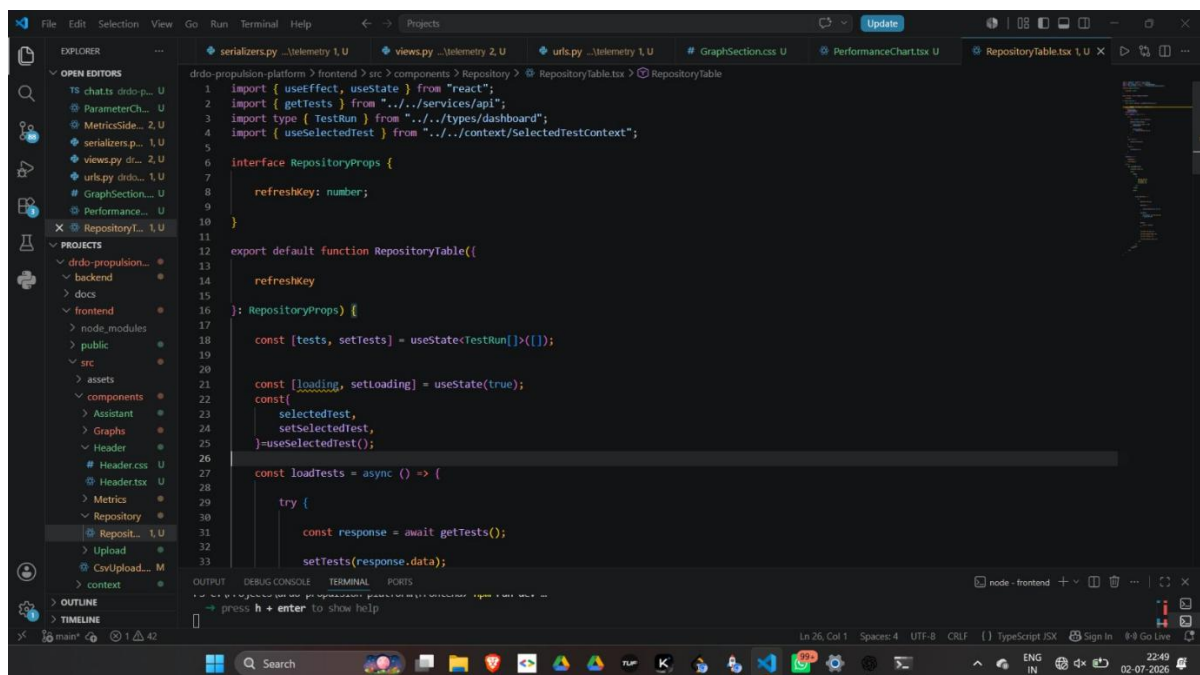


Figure 5.9: React Implementation of the Test Repository Module

### 5.10 AI Engineering Assistant

The AI Assistant serves as the primary interaction interface between engineers and the intelligent analytical system.

Engineers can ask questions such as:

- Why is engine vibration increasing?
- Explain abnormal fuel flow.
- What causes high EGT?
- Suggest possible engine faults.

The assistant retrieves relevant engineering documents before generating responses.

As a result, recommendations remain technically relevant and consistent with uploaded documentation.

### 5.11 Benefits of RAG-Based AI Assistant

The proposed AI architecture provides several significant advantages.

- Improved response accuracy.
- Reduced hallucination.
- Engineering-specific recommendations.
- Faster document retrieval.
- Offline AI execution.
- Better decision support.
- Secure handling of sensitive engineering data.

These advantages make the proposed AI assistant highly suitable for aerospace engineering applications.

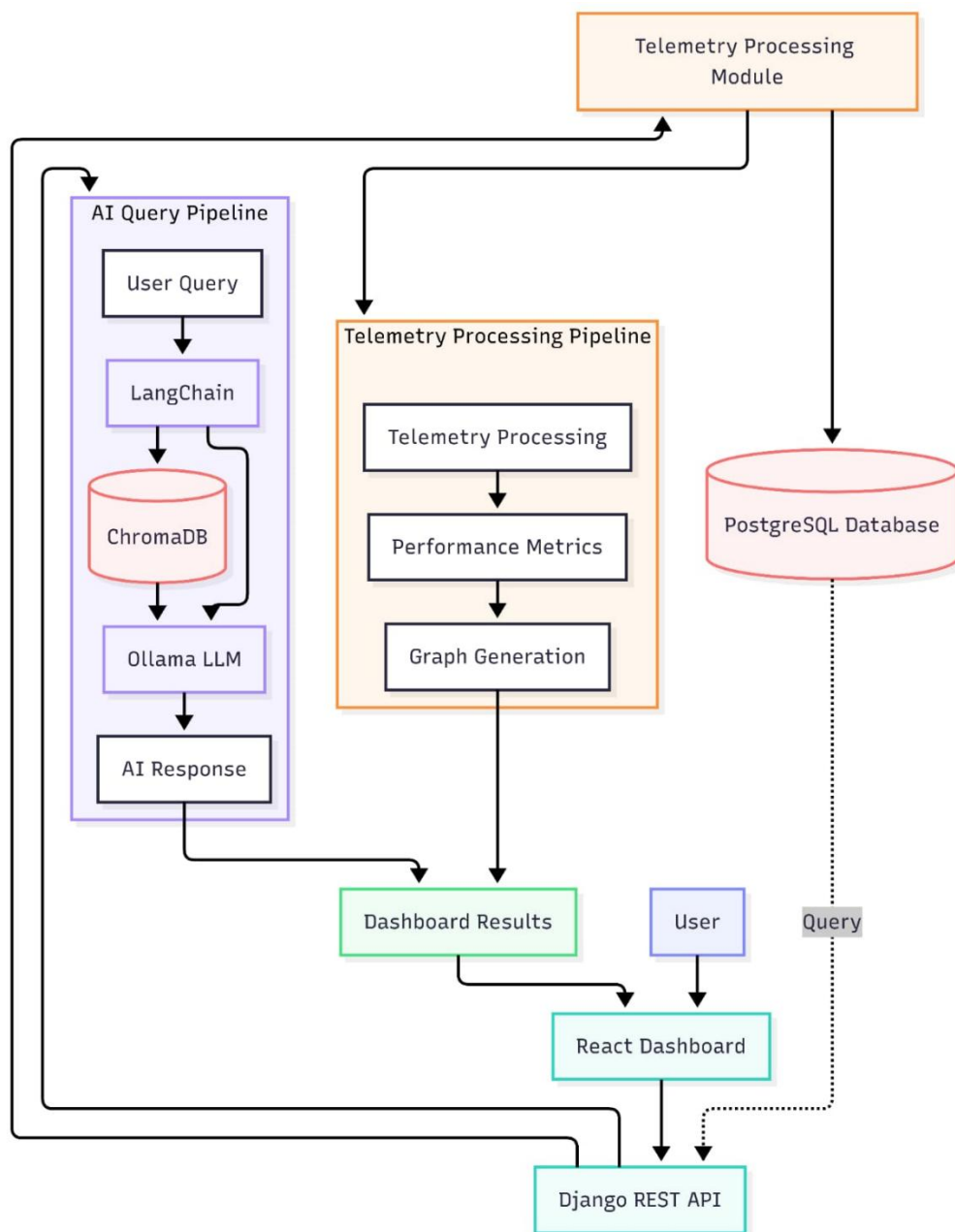
**Table 5.3: Comparison of Traditional AI and RAG-Based AI**

| Feature                    | Traditional AI | RAG-Based AI |
|----------------------------|----------------|--------------|
| Uses Engineering Documents | No             | Yes          |
| Response Accuracy          | Medium         | High         |
| Context Awareness          | Limited        | Excellent    |
| Hallucination              | Higher         | Lower        |

| Feature                     | Traditional AI | RAG-Based AI     |
|-----------------------------|----------------|------------------|
| Technical Reliability       | Moderate       | High             |
| Engineering Recommendations | Generic        | Context-Specific |

### Explanation

The comparison shows that RAG-based AI provides more reliable and technically accurate responses because it incorporates relevant engineering documentation into the response generation process.



Flowchart 5 – Retrieval Augmented Generation (RAG) Workflow

## **CHAPTER 6 - TELEMETRY DATA PROCESSING AND ENGINE HEALTH ANALYSIS**

### **6.1 Introduction**

Telemetry data plays a vital role in propulsion engine testing by providing continuous information about the operational condition of the engine during test execution. Every propulsion engine is equipped with multiple sensors that monitor important parameters such as Exhaust Gas Temperature (EGT), Fuel Flow, N1 RPM, N2 RPM, Torque, Oil Pressure, and Vibration. These sensor readings help engineers evaluate engine performance, identify abnormalities, and verify whether the engine is operating within safe limits.

Traditionally, engineers manually analyze these telemetry values after the completion of a test. This process is time-consuming because each parameter must be examined individually before conclusions can be drawn. As the number of engine tests increases, the amount of telemetry data also grows significantly, making manual analysis increasingly difficult.

The proposed AI-Based Propulsion Test Analysis Platform automates this entire process. Engineers simply upload the telemetry data in CSV format, and the system performs data validation, parameter extraction, performance calculations, graph generation, historical record storage, and engine health evaluation automatically. This automation reduces analysis time, minimizes human error, and provides engineers with meaningful insights through an interactive dashboard.

This chapter explains the telemetry processing pipeline, engine health analysis methodology, and performance visualization implemented in the proposed system.

### **6.2 CSV Upload Module**

#### **Introduction**

The first stage of the telemetry analysis process is the CSV Upload Module. This module allows engineers to upload telemetry datasets collected during propulsion engine testing. The uploaded CSV file contains sensor readings recorded throughout the engine test cycle.

The upload interface has been designed to be simple and user-friendly. Engineers can select the required telemetry file, submit it to the system, and immediately begin the analysis process.

After the upload is completed, the backend verifies the file format and forwards it to the telemetry processing module.

#### **Functions of the Upload Module**

The CSV Upload Module performs several important functions.



- Accepts telemetry datasets.
- Validates the uploaded file.
- Checks file integrity.
- Transfers data to the backend.
- Initiates telemetry processing.

This module serves as the entry point of the complete analytical workflow.

### 6.3 Telemetry Data Validation

#### Introduction

Before telemetry analysis begins, the uploaded CSV file undergoes a validation process to ensure that the data is complete, consistent, and free from major formatting errors.

The validation module checks whether all required engine parameters are available. It also verifies that the uploaded dataset follows the expected column structure.

If missing values or incorrect formats are detected, the system generates appropriate validation messages to prevent incorrect analysis.

#### Validation Activities

The validation module performs the following checks:

- CSV file format verification.
- Required column validation.
- Missing value detection.
- Duplicate record identification.
- Data consistency verification.

These validation steps improve the reliability of subsequent analytical calculations.

**Table 6.1: CSV Validation Process**

| Validation Step         | Purpose                          |
|-------------------------|----------------------------------|
| File Format Check       | Verify CSV format                |
| Column Verification     | Ensure required parameters exist |
| Missing Value Detection | Identify incomplete data         |

| Validation Step      | Purpose                 |
|----------------------|-------------------------|
| Duplicate Check      | Remove repeated records |
| Data Integrity Check | Maintain consistency    |

## 6.4 Telemetry Data Processing

### Introduction

After successful validation, the telemetry dataset is processed using the Pandas library. Pandas converts the uploaded CSV file into a structured DataFrame that enables efficient numerical analysis.

Each sensor parameter is extracted separately and prepared for performance calculations. The backend computes summary statistics such as average values, minimum values, maximum values, and operational trends.

The processed information is then stored inside the PostgreSQL database for future reference.

### Processing Steps

The telemetry processing module performs the following operations:

- Read CSV file.
- Extract engine parameters.
- Convert data into DataFrame.
- Clean unnecessary values.
- Perform calculations.
- Store processed results.

This structured processing pipeline ensures that all telemetry data is analyzed consistently.

## 6.5 Engine Performance Parameters

### Introduction

The uploaded telemetry dataset contains several important engine parameters that describe the operational condition of the propulsion system. These parameters are continuously monitored to determine engine efficiency and detect abnormal operating conditions.

Each parameter provides different information regarding engine performance.

**Table 6.2: Major Engine Parameters**

| Parameter | Description               | Unit  |
|-----------|---------------------------|-------|
| EGT       | Exhaust Gas Temperature   | °C    |
| N1 RPM    | Low Pressure Rotor Speed  | RPM   |
| N2 RPM    | High Pressure Rotor Speed | RPM   |
| Fuel Flow | Fuel Consumption Rate     | kg/hr |
| Torque    | Rotational Force          | Nm    |
| Vibration | Engine Oscillation Level  | mm/s  |

### **Explanation**

The above parameters are continuously monitored throughout engine testing. Any significant deviation from normal operating limits may indicate potential engine faults or abnormal operating conditions.

## **6.6 Engine Health Score Calculation**

### **Introduction**

One of the important features of the proposed system is the automatic calculation of the Engine Health Score.

Instead of manually interpreting multiple telemetry parameters, the platform summarizes engine performance using a single health indicator.

The health score is calculated after evaluating several operational parameters, including temperature, rotational speed, vibration, and fuel consumption.

A higher health score indicates stable engine performance, while lower values may require further engineering investigation.

This simplified evaluation enables engineers to assess engine condition more quickly

### **Benefits of Health Score**

- Simplifies performance interpretation.
- Reduces manual calculations.
- Supports rapid decision-making.
- Improves operational efficiency.

## **6.7 Performance Graph Generation**

### **Introduction**

Graphical visualization is one of the most effective methods for understanding engine behaviour.

Instead of examining large numerical tables, engineers can observe parameter variations using interactive graphs.

The system automatically generates graphs representing:

- Exhaust Gas Temperature.
- Fuel Flow.
- N1 RPM.
- N2 RPM.
- Torque.
- Vibration.

These visualizations help engineers identify abnormal trends much more efficiently.

## **6.8 Historical Test Repository**

### **Introduction**

Every processed engine test is stored within the Historical Test Repository.

The repository allows engineers to review previous telemetry datasets, compare engine performance across multiple test sessions, and retrieve historical information whenever required.

Maintaining historical records improves long-term performance monitoring and supports engineering investigations.

### **Repository Features**

- Stores completed engine tests.
- Retrieves previous telemetry data.
- Supports historical comparison.
- Maintains analytical reports.

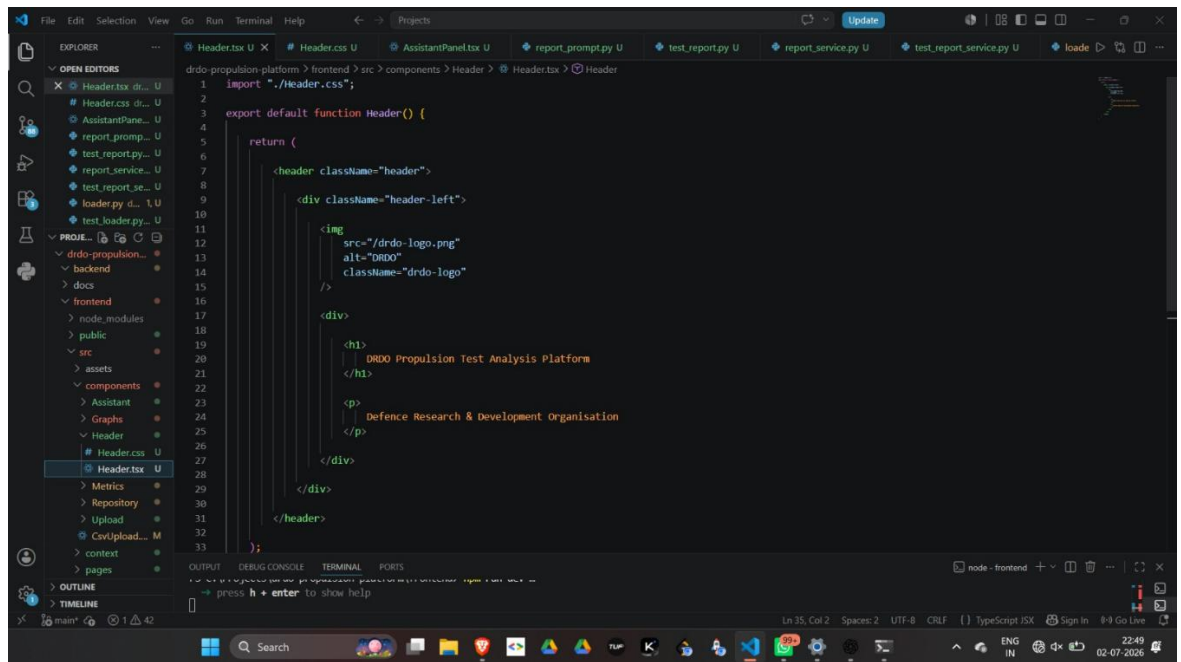
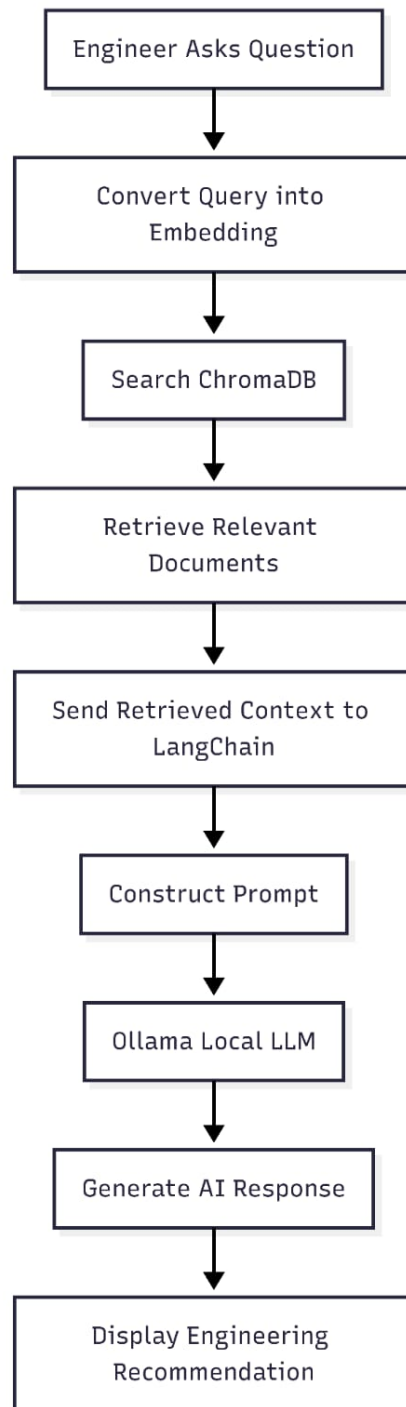


Figure 6.5: AI Assistant Interface with Test Repository



Flowchart 6- Telemetry Data Processing Pipeline

## **CHAPTER 7 - IMPLEMENTATION AND DASHBOARD DEVELOPMENT**

### **7.1 Introduction**

The implementation phase is one of the most important stages in software development because it transforms the designed architecture into a fully functional application. During this phase, all software modules were developed, integrated, and tested to ensure that the proposed system performs the required operations efficiently. The AI-Based Propulsion Test Analysis Platform was implemented using a modern full-stack development approach, combining ReactJS for the frontend, Django REST Framework for backend services, PostgreSQL for database management, and Artificial Intelligence technologies such as LangChain, ChromaDB, and Ollama.

The implementation focused on creating an interactive dashboard that enables engineers to upload telemetry files, monitor engine performance, retrieve historical test records, and communicate with an AI assistant through a single interface. Each module was developed independently and later integrated using REST APIs to provide smooth communication between the frontend and backend.

The final implementation successfully achieved all project objectives by automating telemetry analysis, graph generation, AI-assisted engineering support, and historical data management while maintaining an intuitive and user-friendly interface.

### **7.2 Frontend Dashboard Development**

#### **Introduction**

The frontend of the proposed platform was developed using ReactJS, a modern JavaScript library known for its component-based architecture and dynamic rendering capabilities. The dashboard was designed with simplicity and usability in mind so that engineers can perform all major operations from a single screen.

The dashboard provides access to telemetry upload, quick performance metrics, recent test records, AI interaction, and graphical analysis. React components were developed separately for each module, making the application modular, reusable, and easier to maintain.

The responsive design ensures that the interface remains organized and accessible while displaying large amounts of analytical information.

## Major Dashboard Components

The dashboard consists of the following components:

- Navigation Header
- CSV Upload Module
- Quick Metrics Panel
- Recent Test Repository
- Performance Graph Section
- AI Assistant Panel

Each component communicates with the backend using REST APIs to display real-time information.

### 7.3 CSV Upload and Data Submission Module

#### Introduction

The CSV Upload Module enables engineers to submit propulsion engine telemetry datasets for analysis. A simple upload interface allows users to select a CSV file from the local system and submit it to the backend.

After the upload request is received, the backend validates the file, extracts telemetry values, stores the information in the database, and begins the analysis process automatically.

The upload module minimizes manual effort while ensuring that telemetry datasets are processed efficiently.

#### Functions of Upload Module

- Select telemetry CSV file.
- Validate uploaded file.
- Transfer data to backend.
- Trigger processing pipeline.
- Display upload status.



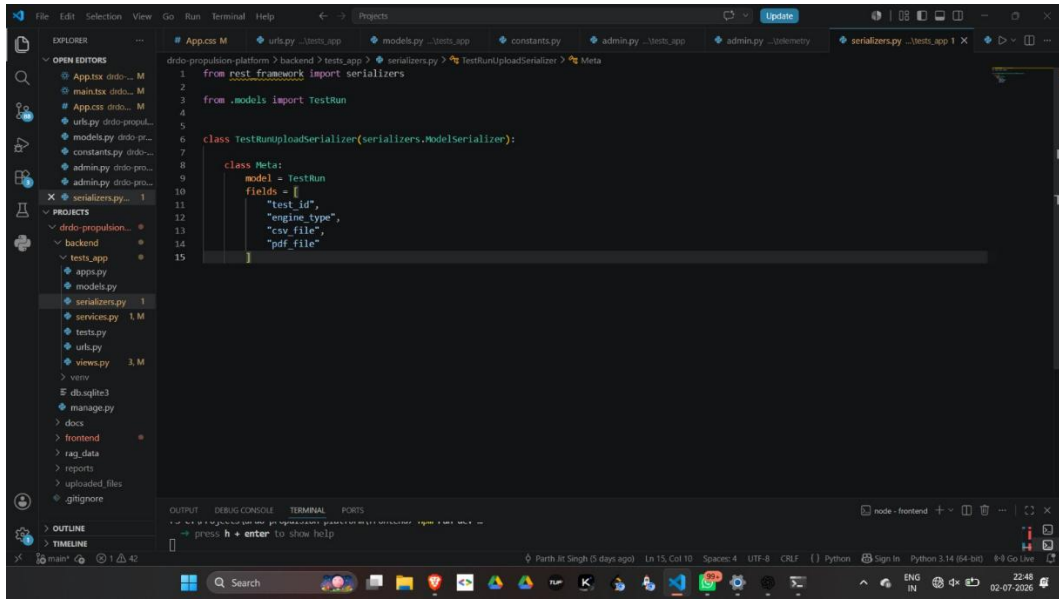


Figure 7.3- Telemetry Performance Graphs Generated by Analysis System

## 7.4 Backend Processing Implementation

### Introduction

The backend was implemented using Python and Django REST Framework. It performs all computational tasks within the platform, including telemetry processing, database operations, graph generation, and AI communication.

Whenever the frontend sends a request, the backend receives the request, processes the data, stores the results, and returns structured responses to the dashboard.

The modular design of the backend improves maintainability and allows future enhancements without major architectural modifications.

### Backend Responsibilities

The backend performs the following functions:

- CSV processing
- Data validation
- Metric calculation
- Database communication
- Graph generation
- AI request handling
- Historical record management

## **7.5 Historical Test Repository**

### **Introduction**

The Historical Test Repository stores all processed propulsion engine test records for future reference. Every completed test is automatically saved in the database, allowing engineers to retrieve previous analyses whenever required.

The repository provides easy access to historical data, supporting long-term performance monitoring and comparative analysis between multiple test sessions.

This feature improves traceability and reduces the need to repeat analytical procedures.

### **Repository Features**

- Store processed engine tests.
- Retrieve historical records.
- Display test summaries.
- Support comparative analysis.

## **7.6 Performance Graph Dashboard**

### **Introduction**

Graphical representation of telemetry data enables engineers to understand engine behaviour more effectively than numerical values alone. The system automatically generates graphs representing important engine parameters immediately after telemetry processing.

Graphs included in the dashboard are:

- Exhaust Gas Temperature (EGT)
- Fuel Flow
- N1 RPM
- N2 RPM
- Torque
- Vibration

These graphs help engineers identify trends, abnormal behaviour, and parameter fluctuations throughout the engine test.

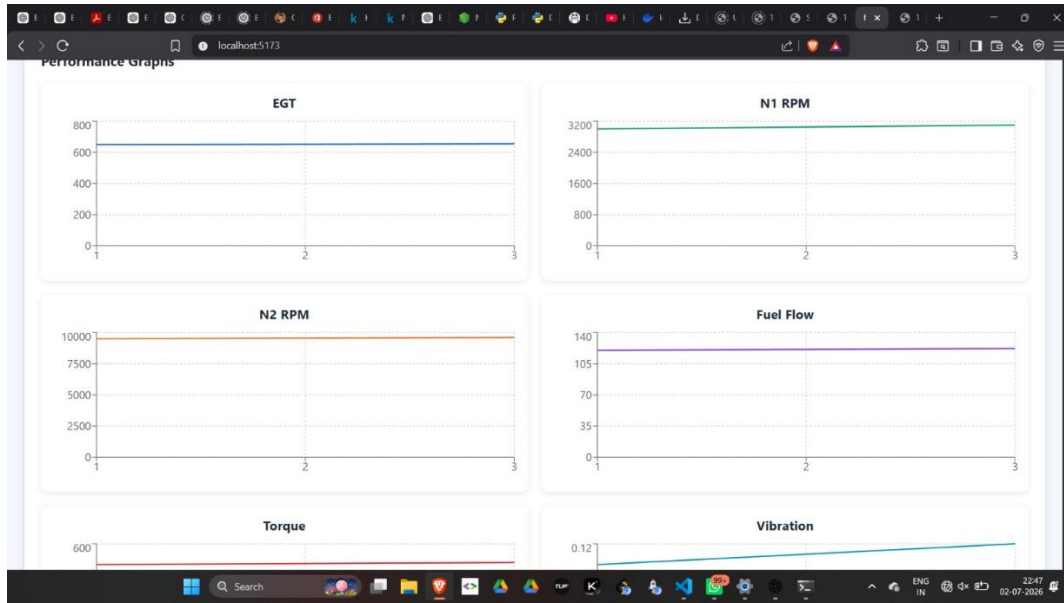


Figure 7.6: Performance Graph Dashboard

Table 7.1: Dashboard Graphs

| Graph     | Purpose                      |
|-----------|------------------------------|
| EGT       | Monitor engine temperature   |
| Fuel Flow | Measure fuel consumption     |
| N1 RPM    | Monitor low-pressure rotor   |
| N2 RPM    | Monitor high-pressure rotor  |
| Torque    | Evaluate rotational force    |
| Vibration | Detect abnormal oscillations |

## 7.7 AI Assistant Implementation

### Introduction

One of the most innovative features of the proposed platform is the integration of the **AI** Engineering Assistant. The assistant enables engineers to ask technical questions related to propulsion engine performance and receive context-aware recommendations.

The AI assistant retrieves relevant engineering documents using Retrieval Augmented Generation (RAG) before generating responses through the locally deployed Ollama Large Language Model.

This implementation provides technically relevant answers while ensuring that sensitive engineering information remains within the local system.

**AI Assistant Features**

- Technical question answering.
- Engineering document retrieval.
- Context-aware responses.
- Offline AI execution.
- Engineering recommendation generation.

**7.8 Module Integration**

**Introduction**

The success of the platform depends on the seamless integration of all software modules. Each module communicates with others through REST APIs, ensuring efficient data transfer and synchronized operation.

The integration process begins with the upload of telemetry data. The backend validates and processes the information, stores the results in PostgreSQL, generates performance graphs, and updates the dashboard. If an engineer submits a technical query, the AI assistant retrieves relevant engineering documents from ChromaDB and generates responses using Ollama.

This integrated workflow provides a smooth user experience while maintaining high processing efficiency.

**Table 7.2: Software Module Integration**

| Module    | Connected Module |
|-----------|------------------|
| ReactJS   | Django Backend   |
| Django    | PostgreSQL       |
| Django    | Pandas           |
| Django    | LangChain        |
| LangChain | ChromaDB         |
| ChromaDB  | Ollama           |

| Module | Connected Module |
|--------|------------------|
| Ollama | AI Assistant     |

### 7.9 Advantages of the Implemented System

The implemented software platform provides several advantages over conventional propulsion analysis methods.

- Interactive user interface.
- Automated telemetry processing.
- Intelligent AI recommendations.
- Dynamic graph generation.
- Historical test management.
- Offline AI execution.
- Secure engineering data processing.
- Modular software architecture.
- Easy future scalability.

These features collectively improve the efficiency and usability of propulsion engine analysis.

## **CHAPTER 8- TESTING, RESULTS AND PERFORMANCE EVALUATION**

### **8.1 Introduction**

Testing is one of the most important phases in the software development life cycle because it ensures that the developed application performs according to the specified requirements. A well-tested system provides confidence that all modules function correctly, interact properly with one another, and produce reliable outputs under different operating conditions.

The AI-Based Propulsion Test Analysis Platform was tested after the completion of all development activities. Each module, including telemetry upload, CSV processing, database storage, graph generation, historical repository management, and the AI assistant, was verified individually before performing complete system testing.

The objective of testing was to ensure that the platform processes telemetry data accurately, generates meaningful performance metrics, stores historical records correctly, and provides reliable AI-assisted responses. Various test cases were executed to evaluate system functionality, performance, reliability, and usability.

The results obtained during testing demonstrate that the developed platform successfully meets the objectives defined at the beginning of the project.

### **8.2 Testing Methodology**

#### **Introduction**

A systematic testing methodology was followed to verify the performance of the proposed system. Testing was performed at different levels to identify errors during development and to ensure smooth interaction between all software modules.

Initially, individual modules were tested independently. After successful module verification, integration testing was carried out to confirm communication between the frontend, backend, database, and AI components. Finally, complete system testing was conducted using sample propulsion engine telemetry datasets.

The testing process ensured that the application behaved correctly under normal operating conditions and produced consistent analytical results.

#### **Levels of Testing**

The following testing techniques were performed:

- Unit Testing

- Integration Testing
- System Testing
- Functional Testing
- User Interface Testing

Each level focused on verifying different aspects of the application.

**Table 8.1: Types of Testing**

| Testing Type        | Purpose                        |
|---------------------|--------------------------------|
| Unit Testing        | Verify individual modules      |
| Integration Testing | Verify module interaction      |
| System Testing      | Validate complete application  |
| Functional Testing  | Verify required functions      |
| UI Testing          | Check user interface behaviour |

### 8.3 Functional Testing

#### Introduction

Functional testing verifies whether every feature of the application performs according to the specified requirements. Each module was executed with valid input data to confirm correct functionality.

The dashboard successfully accepted telemetry CSV files, initiated backend processing, generated performance metrics, displayed graphs, stored historical records, and enabled communication with the AI assistant.

The observed outputs matched the expected behaviour, indicating that the developed modules function correctly.

**Table 8.2: Functional Test Cases**

| Test Case            | Expected Result              | Status |
|----------------------|------------------------------|--------|
| CSV Upload           | File uploaded successfully   | Passed |
| Data Validation      | Invalid files rejected       | Passed |
| Telemetry Processing | Metrics generated            | Passed |
| Graph Generation     | Graphs displayed             | Passed |
| Repository Storage   | Records stored correctly     | Passed |
| AI Assistant         | Technical response generated | Passed |

### **Explanation**

The functional testing results demonstrate that all major modules of the platform operate successfully and satisfy the intended functional requirements.

## **8.4 User Interface Testing**

### **Introduction**

The user interface was evaluated to ensure that engineers can easily navigate the application and perform the required operations. The dashboard layout was designed to be simple, organized, and responsive.

Different interface components such as buttons, menus, upload controls, graphs, and AI interaction panels were tested for proper operation.

The dashboard responded correctly to user interactions and displayed analytical results without noticeable delays.

## **8.5 Performance Evaluation**

### **Introduction**

Performance evaluation was conducted to determine whether the proposed platform processes telemetry datasets efficiently and responds within acceptable time limits.

The evaluation considered factors such as:

- File upload speed
- CSV processing time
- Graph generation
- Database retrieval



- AI response generation

The system demonstrated stable performance during testing and processed telemetry files within a reasonable time.

The locally deployed AI assistant also generated responses efficiently without requiring internet connectivity.

**Table 8.3: Performance Evaluation**

| Parameter           | Observation   |
|---------------------|---------------|
| CSV Upload          | Successful    |
| Data Processing     | Fast          |
| Graph Generation    | Immediate     |
| Database Access     | Efficient     |
| AI Response         | Context-aware |
| Overall Performance | Satisfactory  |

## 8.6 Result Analysis

### Introduction

The testing results indicate that the proposed platform successfully performs all major analytical tasks required for propulsion engine telemetry analysis.

The dashboard correctly accepted telemetry data, validated uploaded files, generated performance metrics, stored historical records, displayed graphical visualizations, and produced AI-assisted engineering recommendations.

The implementation of Retrieval Augmented Generation improved the quality of AI responses by incorporating engineering documentation into the response generation process.

Overall, the developed platform achieved the intended objectives of reducing manual effort, improving analysis speed, and enhancing engineering decision support.

### Major Results Obtained

The following outcomes were achieved:

- Successful telemetry data processing.

- Accurate performance calculations.
- Automatic graph generation.
- Efficient historical data management.
- Reliable AI-assisted recommendations.
- Offline execution using Ollama.
- Improved analytical efficiency.

### **8.7 Advantages Observed During Testing**

The testing phase demonstrated several practical advantages of the developed system.

- Reduced manual analysis time.
- Faster telemetry processing.
- Improved visualization.
- Better historical record management.
- Secure offline AI operation.
- Context-aware engineering assistance.
- Easy user interaction.
- Scalable software architecture.

These observations confirm that the proposed system provides significant improvements over conventional propulsion analysis methods.

## **CHAPTER 9 - CONCLUSION AND FUTURE SCOPE**

### **9.1 Introduction**

The rapid advancement of Artificial Intelligence (AI), Machine Learning (ML), and Large Language Models (LLMs) has significantly transformed the way engineering data is processed and analyzed. Modern propulsion engine testing generates a large amount of telemetry data that requires accurate interpretation to evaluate engine performance and reliability. Traditional methods of analyzing these datasets involve considerable manual effort, making the process time-consuming and susceptible to human error.

The AI-Based Propulsion Test Analysis Platform was developed to address these challenges by integrating modern software technologies with intelligent data analysis techniques. The platform combines telemetry processing, graphical visualization, historical record management, and AI-assisted engineering support into a single integrated system. Through the implementation of Retrieval Augmented Generation (RAG), LangChain, ChromaDB, and Ollama, the platform provides context-aware technical recommendations while maintaining complete offline operation.

This chapter summarizes the work carried out during the project, evaluates whether the objectives were achieved, highlights the major contributions of the developed system, and discusses possible future enhancements.

### **9.2 Project Summary**

The primary objective of this internship project was to develop an intelligent software platform capable of automating propulsion engine telemetry analysis and assisting engineers during engine performance evaluation.

The project successfully integrated multiple software technologies including ReactJS, Django REST Framework, Python, PostgreSQL, Pandas, Matplotlib, LangChain, ChromaDB, and Ollama into a unified architecture.

The developed platform allows engineers to upload telemetry data in CSV format through a web-based dashboard. After upload, the system automatically validates the data, extracts engine parameters, calculates performance metrics, generates graphical visualizations, stores historical records, and provides AI-assisted technical recommendations.

The implementation of Retrieval Augmented Generation significantly enhanced the quality of AI responses by enabling the assistant to retrieve relevant engineering information before generating answers. Since the Large Language Model operates locally using Ollama, sensitive engineering information remains secure and does not require cloud connectivity.

The completed platform demonstrates how Artificial Intelligence can improve propulsion engine testing by reducing manual effort, improving analysis speed, and supporting engineering decision-making.

### 9.3 Achievement of Project Objectives

At the beginning of the project, several objectives were defined to guide the development process. After implementation and testing, it was observed that all major objectives were successfully achieved.

The web-based dashboard was developed successfully and provides an interactive environment for engineers to perform telemetry analysis. The platform supports CSV file upload, automated telemetry processing, graphical visualization, and historical record management.

The AI Engineering Assistant was successfully integrated using Retrieval Augmented Generation, allowing engineers to ask technical questions and receive context-aware responses based on engineering documentation.

The platform also demonstrated secure offline AI execution through Ollama, making it suitable for applications where data confidentiality is important.

Overall, the developed software fulfills the intended functional requirements and achieves the objectives established during the planning phase.

**Table 9.1: Project Objectives and Achievement Status**

| Objective                     | Status   |
|-------------------------------|----------|
| Develop Interactive Dashboard | Achieved |
| CSV Telemetry Upload          | Achieved |
| Automated Data Processing     | Achieved |
| Engine Performance Analysis   | Achieved |
| Graph Generation              | Achieved |
| Historical Repository         | Achieved |
| AI Engineering Assistant      | Achieved |
| RAG Implementation            | Achieved |
| Local LLM Deployment          | Achieved |
| Automated Report Support      | Achieved |

## **Explanation**

The table shows that all major project objectives were successfully implemented and validated during testing. This confirms that the proposed platform meets the expected functional and technical requirements.

### **9.4 Major Contributions of the Project**

The project introduces several significant improvements over conventional propulsion engine analysis methods.

One of the major contributions is the automation of telemetry data processing. Engineers are no longer required to manually inspect large CSV datasets, as the system performs data validation, processing, and visualization automatically.

Another important contribution is the integration of Artificial Intelligence with engineering documentation through Retrieval Augmented Generation. This enables the AI assistant to provide technically relevant responses instead of generic answers.

The implementation of a local Large Language Model using Ollama enhances security by ensuring that engineering information remains within the local environment.

The project also demonstrates the practical application of modern AI technologies within aerospace engineering workflows.

#### **Major Contributions**

- Automated telemetry processing.
- Interactive analytical dashboard.
- AI-assisted engineering recommendations.
- Retrieval Augmented Generation implementation.
- Offline AI execution.
- Historical test management.
- Graphical performance visualization.
- Modular software architecture.

### **9.5 Benefits of the Developed System**

The proposed platform provides several practical benefits for propulsion engine analysis.

The automation of telemetry processing significantly reduces the amount of manual work required during engine testing. Engineers can obtain analytical results within a much shorter time compared to conventional methods.

Interactive graphs improve the understanding of engine behaviour by presenting telemetry information visually. Historical record management allows engineers to compare multiple engine tests and monitor long-term performance.

The AI Engineering Assistant further improves productivity by answering technical questions and retrieving relevant engineering knowledge automatically.

The modular architecture also supports future software expansion without major redevelopment.

**Table 9.2: Benefits of the Proposed Platform**

| Benefit              | Description                      |
|----------------------|----------------------------------|
| Faster Analysis      | Reduced processing time          |
| Improved Accuracy    | Automated calculations           |
| Better Visualization | Interactive graphs               |
| AI Support           | Intelligent recommendations      |
| Data Security        | Local AI execution               |
| Historical Records   | Easy retrieval of previous tests |
| Scalability          | Easy future expansion            |

## 9.6 Future Scope

Although the developed platform successfully achieves its intended objectives, several opportunities exist for further improvement.

Future versions of the system can support additional telemetry formats such as Excel, JSON, and real-time sensor streams, enabling broader compatibility with different testing environments.

The platform can also be enhanced by integrating predictive maintenance algorithms that forecast component failures before they occur. Such capabilities would improve engine reliability and reduce maintenance costs.

Cloud deployment may also be considered for organizations that require multi-user access, centralized data management, and remote monitoring capabilities while maintaining appropriate security measures.

Future AI enhancements may include support for multilingual engineering assistance, voice-based interaction, automatic anomaly detection using deep learning, and integration with live telemetry acquisition systems.

These enhancements would further increase the usefulness of the platform in aerospace and defense industries.

## **CHAPTER 10 - REFERENCES AND BIBLIOGRAPHY**

### **10.1 Introduction**

The successful completion of the AI-Based Propulsion Test Analysis Platform involved studying various technologies, programming frameworks, research articles, official documentation, and technical resources. These references provided valuable information regarding Artificial Intelligence, Retrieval Augmented Generation (RAG), web application development, database management, telemetry data processing, and software implementation.

The following books, research papers, and official documentation were referred to during the design and development of the project.

### **10.2 Books**

1. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, Deep Learning, MIT Press, 2016.
2. Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, 4th Edition, Pearson, 2021.
3. Wes McKinney, Python for Data Analysis, 3rd Edition, O'Reilly Media, 2022.
4. Luciano Ramalho, Fluent Python, 2nd Edition, O'Reilly Media, 2022.
5. Martin Kleppmann, Designing Data-Intensive Applications, O'Reilly Media, 2017.
6. Eric Matthes, Python Crash Course, No Starch Press, 2023.

### **10.3 Research Papers**

1. Lewis, Patrick et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, NeurIPS, 2020.
2. Vaswani, Ashish et al., Attention Is All You Need, NeurIPS, 2017.
3. Devlin, Jacob et al., BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, NAACL, 2019.
4. Brown, Tom et al., Language Models are Few-Shot Learners, NeurIPS, 2020.
5. OpenAI, GPT Models Technical Overview, 2024.

### **10.4 Official Documentation**

The following official documentation was referred to during software development:

- ReactJS Official Documentation



- Django Official Documentation
- Django REST Framework Documentation
- Python Official Documentation
- PostgreSQL Documentation
- Pandas Documentation
- Matplotlib Documentation
- LangChain Documentation
- ChromaDB Documentation
- Ollama Documentation

### **10.5 Online Learning Resources**

The following educational platforms were used for learning and implementation purposes:

- GeeksforGeeks
- W3Schools
- MDN Web Docs
- Stack Overflow
- GitHub Documentation
- Medium Technical Articles

### **10.6 Websites Referenced**

- Python Official Website
- React Official Website
- Django Official Website
- PostgreSQL Official Website
- Pandas Official Website
- Matplotlib Official Website
- LangChain Official Website
- Chroma Official Website
- Ollama Official Website

- GitHub

## **10.7 Bibliography**

The development of this project was supported by books, official documentation, research papers, online technical articles, and practical implementation using modern software development tools. These references provided the theoretical knowledge and technical guidance necessary for implementing the AI-Based Propulsion Test Analysis Platform.

The combination of academic literature and official software documentation ensured that the project followed current software engineering practices and incorporated modern Artificial Intelligence technologies suitable for aerospace telemetry analysis.